

SkillBridge AI: Intelligent Career Development Platform for College Students

Project Proposal for UCS503P
Submitted to: Ms.Paramveer Kaur
Thapar Institute of Engineering and Technology

Uday Dhir, COE* Aishwarya Verma, COE†

August 13, 2026

1 Higher-order goal

... secondary framing

This project aims to improve the results of student employability by reducing the friction, guesswork, and fragmentation students face when preparing for campus placements. Although the underlying problem — career readiness — is broad, the immediate engineering objective is to translate that need into a measurable, deployable software platform that gives a student a single place to assess, improve and track their placement readiness, built around one coherent, fully working core rather than a wide but shallow feature set.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by building the schema and authentication first, then one feature module at a time, validating each against seeded resume/job data before moving to the next.
- We will prioritize fast feedback over breadth by using weekly iterations, with CI-backed tests required to pass after each module before the next module begins.
- Success is defined by what can be delivered, tested, and demonstrated within the semester: a fully working resume-to-readiness pipeline, not a partially-implemented version of every feature imaginable for the platform.

3 Problem Statement

Final-year and pre-final-year students preparing for placements typically juggle disconnected resources: informal resume reviews, separate coding-practice sites, ad hoc or nonexistent interview practice, and no single view of how “placement-ready” they actually are. Common pain points include:

- **Fragmentation:** Resume feedback, coding practice, aptitude prep, and interview practice live in separate, disconnected tools.

*Roll No: 1024030168, Email: udhir.be24@thapar.edu

†Roll No: 1024030170, Email: averma4.be24@thapar.edu

- **No objective benchmark:** Students cannot easily tell how their resume or preparation compares against what recruiters actually screen for.
- **Delayed, generic feedback:** Resume and interview feedback, when available, is qualitative, inconsistent, and slow to arrive.
- **Recruiter-side inefficiency:** Recruiters and campus placement cells manually screen large volumes of resumes with limited tooling to filter or rank candidates.

Why this matters now (contextual relevance): With placement cycles compressed into a few weeks and many students competing for a limited number of slots, even a modest reduction in feedback delay and an increase in the precision of self-assessment can measurably improve how well-prepared a student is by the time they sit for an interview.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a **web-based** “SkillBridge” platform with the following components, each built completely rather than left partially implemented:

- **Resume analyzer:** Parses PDF/DOCX resumes, extracts skills/education/projects/experience via NLP, computes an explainable ATS score, and flags missing keywords and weak sections.
- **Skill-gap and roadmap engine:** Compares a student’s profile against one of three pre-defined career tracks and generates a prioritized skill list, weekly study plan, and curated resource/certification suggestions.
- **Coding assessment module:** A curated question bank (single language) with auto-evaluated submissions, hidden test cases, and a leaderboard.
- **Placement Readiness Score:** A documented, weighted composite of resume, coding, aptitude, and mock-interview scores.
- **Text-based AI mock interview:** A fixed technical question bank per track, with LLM-based rubric scoring of typed answers and a generated feedback report.
- **Recruiter portal:** Job postings, candidate search/filtering, and AI-assisted candidate ranking using embedding similarity between job requirements and candidate profiles.
- **Admin dashboard:** Platform-wide analytics (users, resumes, jobs, top skills) and basic content management.

4.2 Core workflow

1. Student registers, completes a profile, and uploads a resume.
2. System parses the resume, computes an ATS score, and surfaces missing keywords and weak sections.
3. Student attempts the coding assessment and, optionally, the mock interview; scores feed into a combined Placement Readiness Score.
4. Student selects a career track; the system generates a skill-gap report and roadmap.
5. Recruiters post jobs, search/filter the candidate pool, and view an AI-ranked shortlist.

4.3 Operational constraints for an educational, single semester setting

- Keep the solution usable on low-to-mid range devices via a responsive web frontend.
- Favor well-understood, off-the-shelf NLP/ML components (spaCy, Sentence-Transformers) over building or training novel models from scratch.
- Use an existing LLM API for interview-answer evaluation rather than building a custom evaluation model; the engineering contribution is in prompt/rubric design and structured-output handling, not model training..
- Design for deployability using common web hosting, a managed relational database, and containerized deployment.
- Explicitly exclude team-project matching, voice/video interviews, multi-language code execution, live external job-market data, salary-trend prediction, and calendar/scheduling integration from this semester’s build — see Section 9.3 for the reasoning.

5 Solution Approach

... Engineering focus

This is an engineering course project, so the emphasis is on integrating mature, well-documented components into a coherent, correctly-functioning workflow, rather than on original ML research.

5.1 Resume parsing and ATS scoring

... non-research

Duplicate detection will be heuristic-based:

- Extract text from PDF/DOCX using established libraries (PyMuPDF, python-docx) rather than custom parsers.
- Use standard NLP techniques (spaCy for skill/entity extraction against a curated skills taxonomy, and Sentence-Transformer embeddings with cosine similarity for resume–job-description comparison) without claiming state-of-the-art novelty.
- Implement the ATS score as a documented, inspectable formula (keyword coverage, section completeness, formatting signals) rather than an opaque model, so the logic can be explained and defended.
- Present the ATS score and missing-keyword list as decision-support for the student; never silently reject a resume.

5.2 Mock interview evaluation

... LLM integration, not model-building

- Use a fixed, curated technical-question bank per career track; the student answers in text.
- Send the question, rubric, and student answer to an LLM API with a structured prompt, and parse the response into a technical-correctness score, a clarity score, and specific improvement tips.
- Store the transcript and scores for a per-attempt report, demonstrating structured LLM-API integration rather than a from-scratch conversational evaluation model.

5.3 Web architecture

... *web-based solution*

- **Frontend:** responsive UI for students, recruiters, and admins (dashboards, upload forms, assessment views).
- **Backend API:** authentication, resume/assessment endpoints, scoring logic, ranking endpoints.
- **Database:** stores users, resumes, skills, assessments, jobs, applications, and analytics.

5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run backend/ML unit tests on every change, including tests for the ATS scoring formula and the coding-assessment evaluator.
- Produce repeatable, containerized deployment artifacts to a staging environment.
- Enable safe, frequent releases as each module (resume analyzer, assessments, roadmap, recruiter portal) is completed.

6 Evaluation Criterion

... *measurable and attributable*

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Time-to-feedback (TTF): time between resume upload and the student receiving an ATS score and skill-gap report.

- Target: median TTF ≤ 30 seconds during pilot testing.
- Attribution: measured from server timestamps (upload event vs. report-generation event).

6.2 Secondary evaluation metrics

- **ATS scoring consistency:** Variance in score for the same resume re-uploaded, used as a sanity check on scoring stability.
- **Assessment completion rate:** percentage of registered students who complete the coding assessment.
- **Ranking relevance:** for a sample of job postings, whether the top AI-ranked candidates match a manual reviewer's top picks (spot-checked, not a formal offline benchmark).
- **User clarity:** student-reported clarity of feedback using a simple 1–5 survey question.
- **Reliability:** availability of staging/production for login, upload, and assessment.

6.3 Pilot validation plan

... *high-level*

- Recruit 15–25 pilot student users and 2–3 simulated recruiter accounts.
- Collect baseline expectations via a short pre-pilot survey (e.g., “how do you currently get resume feedback?”) and compare against in-app metrics after the pilot window.

7 Scalability

... *with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in resume volume, concurrent assessments, and recruiter search load by:
 - decoupling frontend, backend API, and ML scoring services,
 - enabling pagination and indexing for candidate search and analytics queries,
 - using a stateless API design so scoring/embedding workers can be scaled horizontally.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed relational database (PostgreSQL),
 - containerized deployment (Docker) for both the API and ML services,
 - CI/CD pipeline for staging/production.

The architecture (single-language coding judge, curated question banks, three career tracks) is deliberately narrow for this semester, but each of these limits is a configuration/content boundary rather than a structural one — adding a language, a track, or a question is additive, not a rewrite.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs (FastAPI) and a React/Tailwind frontend.
- Managed relational database (PostgreSQL).
- Token-based authentication (JWT) with role-based authorization for student/recruiter/admin roles.
- Established NLP libraries (spaCy, Sentence-Transformers) rather than custom-trained models.
- An existing LLM API (Anthropic/OpenAI) for mock-interview answer evaluation.
- Test framework for backend and ML-pipeline unit/integration tests.
- CI runner and staging deployment target (Docker, GitHub Actions).

We will use these mature components to focus on workflow correctness, data modeling, and measurement, rather than inventing new ML infrastructure.

9 Project scope and deliverables

... *iteration-friendly*

9.1 Initial deliverable

... *first iteration*

- Database schema, migrations, and JWT authentication with role-based access.
- Upload of resume (PDF/DOCX), text extraction, skill extraction, and ATS score generation with missing-keyword report.
- Student dashboard showing resume score.
- CI: build + backend unit tests + linting; CD to staging with a single pipeline job.

9.2 Subsequent deliverables

- Coding assessment module (single language, curated question bank, auto-evaluation, leaderboard).
- Skill-gap analysis and career roadmap generator for three predefined career tracks.
- Placement Readiness Score combining resume, coding, aptitude, and interview scores.
- Text-based AI mock interview with LLM-based rubric scoring and a per-attempt report.
- Recruiter portal: job postings, candidate search/filter, and embedding-based candidate ranking.
- Admin dashboard: platform analytics and basic content management (CRUD on questions/jobs/users).

10 Risks and Mitigations

- **ATS scoring accuracy:** heuristic/embedding-based scoring may not match every resume style; mitigate by testing against a varied seed dataset and treating scores as guidance, not a definitive judgment.
- **LLM dependency for mock interviews:** API cost/latency/availability risk; mitigate by caching rubric prompts, capping attempts per student during the pilot, and handling API failures gracefully in the UI.
- **Resume/data privacy:** minimize retained personal data, keep resume storage access-controlled, and only collect fields the platform actively uses
- **Evaluation measurement ambiguity:** instrument timestamps and define scoring logic before development begins so metrics in Section 6 are attributable..
- **Operational issues in pilot:** use staging early and ensure CI/CD produces repeatable deployments before onboarding pilot users.
- **Timeline pressure:** The strict module ordering in Section 9.2 is to keep the plan achievable within one semester.

11 Summary

This project proposes SkillBridge AI, a deployable web platform that helps students assess and improve their placement readiness through resume analysis, skill-gap detection, a coding assessment, a text-based AI mock interview, and readiness tracking, while giving recruiters a structured, AI-assisted way to discover suitable candidates. It meets the course criteria by emphasizing time-to-value through a strictly sequenced build, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-feedback). It combines established NLP components (spaCy, Sentence-Transformers) with structured LLM-API integration rather than pursuing research-driven novelty, and explicitly excludes lower-value or disproportionately complex features (team matching, voice interviews, multi-language execution, live market data) so that the full scope in Section 9 is realistically achievable within a single semester.